

Now let's go ahead and add the values for all the patients who have recovered from COVID-19 in each state. In order to do this, we will have to add a separate column to our data frame. Figure 5 shows the output of adding the recovered patients' column to the data frame.

```
#Adding a column to data frame
recovered = [1710, 37390, 16395, 10664, 13316, 7501]
df['Recovered'] = recovered
df
```

	States	Confirmed	Recovered
0	Telangana	3496	1710
1	Maharashtra	82968	37390
2	Tamil Nadu	30152	16395
3	Delhi	27654	10664
4	Gujarat	19592	13316
5	Rajasthan	10331	7501

Figure 5: Adding a column to the data frame

Dropping rows and columns

Dropping or deleting rows and columns in a Pandas data frame is easy using the `drop()` method. We can drop a row using its index, observation or using an *if-else* condition. We can even drop a range of rows. Since this is an introductory article, I will not get into the details but will show readers the simplest way to do this. Let us drop the row containing data for Rajasthan. This can be done using the following code. Figure 6 shows the data frame after dropping the row containing Rajasthan.

```
#drop a row from data frame
```

```
df = df.drop(df.index[5])
df
```

	States	Confirmed	Recovered
0	Telangana	3496	1710
1	Maharashtra	82968	37390
2	Tamil Nadu	30152	16395
3	Delhi	27654	10664
4	Gujarat	19592	13316

Figure 6: Dropping a row from the data frame

Now let us drop the 'Recovered' column from our data set. Figure 7 shows the data frame after dropping the 'Recovered' column.

```
#drop a column from data frame
```

```
df = df.drop('Recovered', axis=1)
df
```

The `axis=1` property lets the `drop()` method know that the reference is to a column instead of a row.

	States	Confirmed
0	Telangana	3496
1	Maharashtra	82968
2	Tamil Nadu	30152
3	Delhi	27654
4	Gujarat	19592

Figure 7: Dropping a column from the data frame

Update values

You can update existing values in the rows of the data frame. Again, there are many different ways that you can do this. For the purpose of this tutorial, I am going to show you what I think is the simplest technique, using the `at()` method in Pandas.

Let's say we want to update the value of confirmed cases in Maharashtra from 82,968 to 83,000. We can achieve the output shown in Figure 8 using the following piece of code:

```
#Edit/Update values
```

```
df.at[1, 'Confirmed'] = 83000
df
```

	States	Confirmed
0	Telangana	3496
1	Maharashtra	83000
2	Tamil Nadu	30152
3	Delhi	27654
4	Gujarat	19592

Figure 8: Updating a value in the data frame

Exporting data

You can export your Pandas data frame into multiple file types such as JSON, `csv`, `xlsx`, etc. The following code snippet contains the code for some of the commonly used data types for export:

```
#export data frame to csv
df.to_csv()
```

```
#export data frame to xlsx
df.to_excel()
```

```
#export data frame to JSON
df.to_json()
```

Looking forward

It is important to keep in mind that we have only touched the basics of the Pandas library in this article. There is a whole lot more to this library with a wide array of methods, attributes and advanced techniques of dealing with complex data. My book, 'Data Analytics with Pandas for Absolute Beginners', covers some of the more advanced concepts and methods in detail. **END** 🐧

References

- [1] <https://www.codelatte.site/books/data-analytics-with-pandas>
- [2] https://www.pandas.pydata.org/docs/user_guide/index.html
- [3] https://www.tutorialspoint.com/python_pandas/index.htm

By: Mir H.S. Quadri

The author is a scholar and researcher in the fields of artificial intelligence and machine learning. He has worked on multiple projects using a wide array of frameworks. He is also a FOSS enthusiast and actively contributes to several open source projects. You can find his works on [codelatte.site](https://www.codelatte.site).